

Convex Subdivision

A project for
Computer Geometry
2110597.02
Second Term of 2568

By
Jarukit Jenkriangkrai
Third Year of Chulalongkorn University
GitHub: [Prairiet](#)

Keywords
Triangulation, Convex, Polygon, Subdivision

Abstract

This project aims to take on the problem of subdividing a concave polygon into as few convex polygons as possible. The idea is to triangulate the polygon first, as triangles are always convex. Afterwards, remove edges to form bigger convex polygons. To do this, a variant of a well-known algorithm, the ear-clipping algorithm, was implemented. Compared to simply triangulating the entire polygon, the algorithm reduced the number of edges used, and thus the number of sub-polygons made by about 40%. However, this project's algorithm is still imperfect, and there are further ways to improve it.

The method mentioned above, a variant on an existing algorithm, runs by continuing to clip ears next to already clipped ears and recording them as a single polygon, instead of randomly selecting an ear. This simple variation combines both triangulation and edge reduction into a single step, increasing efficiency.

As for improvements, the data structure itself is basic, and a major limiting factor is the fact that this algorithm is based on a simple algorithm. Sometimes, the algorithm will leave behind edges that can be removed. A half-edge data structure may increase complexity, but in turn allow for more accurate subdivisions.

Table of Contents

Abstract	i
Table of Contents	ii
Problem Definition	1
Baseline	2
Proposed Approach	3
Implementation	6
Experimentation	7
Results	8
Discussion & Conclusion	9
References	10

Problem Definition

The problem is simple. Take a polygon, and subdivide it with as few edges as possible, so that every sub-polygon is convex.

In computer applications, a convex polygon is easier to process than a concave one. Thus, for simplicity, a concave polygon can be thought of as a collection of convex polygons, and each one is then processed individually. This idea can be seen in many places, such as in 3D models, where every object is a collection of triangles, which are always convex. It can also be seen in games, such as fighters, where a hitbox or hurtbox is not a single polygon, but multiple rectangles or circles in 2D, which are convex. For 3D, the equivalent are boxes and spheres, which are also convex.

To put simply, this problem is about creating a general algorithm to turn a concave polygon into a convex one to make processing easier.

Input	Output
A polygon's vertices, ordered counter-clockwise.	A list of edges, zero-indexed, each indicating the start and end vertices.
The polygon must be simple. It must not cross itself at any point, and must not have a hole.	The output may not be perfect. However, it should attempt to minimize the number of edges used.

Some strange cases may happen. While checking for intersections, line segments might end up touching on a vertex, or they could be colinear. For this algorithm, these cases are handled as non-intersecting. As for floating point precision, the error in most cases should be small enough that it's a non-issue.

Baseline

The base algorithm that this project builds off of is a well-known algorithm, the ear-clipping algorithm, also sometimes known as ear trimming. Based on the two ears theorem^[1], every polygon that is not a triangle has at least two ears, which can be removed safely. This algorithm has a time complexity of $O(n^2)$, and a space complexity of $O(n)$. Its simplicity makes it a great point to start with in this project.

Of course, since this is a triangulation algorithm, the output will always have the same number of edges. For a polygon with n vertices, this algorithm will always use $n-3$ edges to subdivide the polygon into $n-2$ convex polygons, which here are always triangles.

The goal, then, is to find a way to tweak the algorithm into removing unnecessary edges.

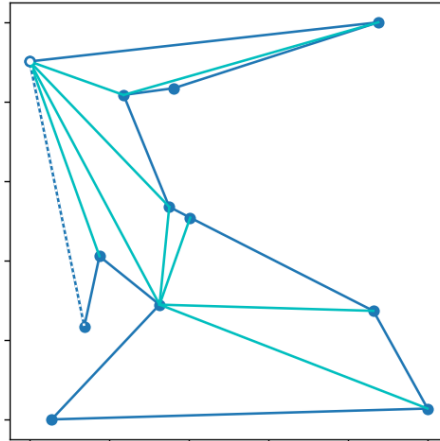


Fig 1.

Triangulation of a dodecagon (12-gon) with the ear-clipping algorithm.

Proposed Approach

Two algorithms are presented here. The first algorithm is quite simple, and it would be more accurate to call it an extension than a proper new algorithm.

All edges that are connected to a convex vertex on both ends are always safe to remove. This is because adding edges always reduces the angle of a vertex and thus cannot cause a convex vertex to become concave. Conversely, removing edges always increases the angle of a vertex, but cannot increase it beyond its original angle. Thus, a vertex that was already convex originally cannot become concave. The goal is to make every vertex convex, so these edges are unnecessary and should be removed.

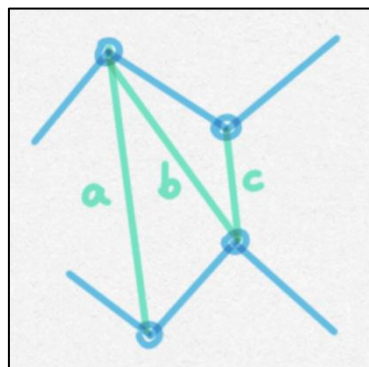


Fig 2.

a is connected to 2 convex vertices and can always be safely removed.

Implementing this is quite simple, and goes something like this:

```
run ear-trimming algorithm
```

```
for e in edges:
```

```
    if e[0] is convex and e[1] is convex:
```

```
        remove e
```

Since it simply builds off the ear-trimming algorithm, its time complexity and space complexity remain the same: $O(n^2)$ and $O(n)$ respectively. This check is $O(n)$ in time complexity.

The second algorithm is not too unlike a flood-fill algorithm, but it also uses the ear-clipping algorithm as its base. Instead of selecting an ear randomly, the clipped ear's neighbors are also clipped if able, and so on until there are no more adjacent ears to be clipped. The last two on either side become the final position for the edge to be placed. Since ears are necessarily convex vertices, we can add them to our working polygon after a simple check.

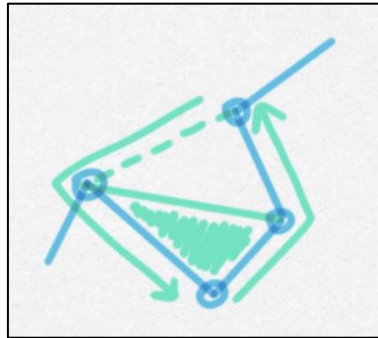


Fig 3.

The dashed edge is added, while the previous edge is removed.

If the new edge would make any of the previous edge's vertices concave, it cannot be added.

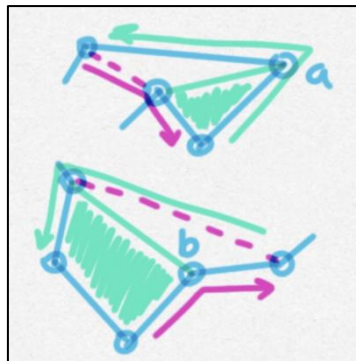


Fig 4.

Neither a nor b can be clipped, as doing so introduces concave vertices.

The algorithm looks like this:

while ears available:

 find ear

 remove ear

 while neighbor can be clipped:

 if neighbor is an ear,

 and it can be clipped without making the working edge's vertices
 concave:

 clip it

 set working edge to ear edge

 add working edge to confirmed edges

Space complexity stays the same at $O(n)$. Time complexity also stays the same at $O(n^2)$, as the validity check is performed in $O(1)$.

Implementation

Both algorithms, as well as the baseline, will be implemented in Python 3.13.5, with 3 libraries being used. `polygenerator`, which as its name implies, is used to generate polygons as input. And the well-known `matplotlib.pyplot` and `numpy`, which are used to display the output and to handle the numbers, such as calculating the matrix determinant for orientation tests, respectively.

GitHub is also used as the project's version control, and the source code and various documents, including this report, can be found in this project's repository.

Experimentation

Both algorithms will be run against a few cases. Using polygenerator, the inputs are convex 50-gons, 20-gons, and 50-gons, and 100-gons. Each case will be taken as the average of 100 trials. Three statistics will be measured: the runtime, the number of edges remaining, and how many times the algorithm crashed due to an edge case, such as floating point errors. Due to limitations, the numbers of errors for the flood fill style algorithm is not available.

Results

Baseline	Runtime (s)	Avg. Edge Count	Errors
20 Concave	0.859	16.98	1
50 Concave	3.156	46.15	19
100 Concave	10.813	91.94	34
50 Convex	4.578	47.00	0

Simple	Runtime (s)	Avg. Edge Count	Errors
20 Concave	0.891	14.88	2
50 Concave	3.094	42.51	10
100 Concave	11.891	86.04	37
50 Convex	5.359	0.00	0

Flood	Runtime (s)	Avg. Edge Count	Errors
20 Concave	0.813	9.58	-
50 Concave	3.063	28.08	-
100 Concave	13.000	57.08	-
50 Convex	4.563	0.00	-

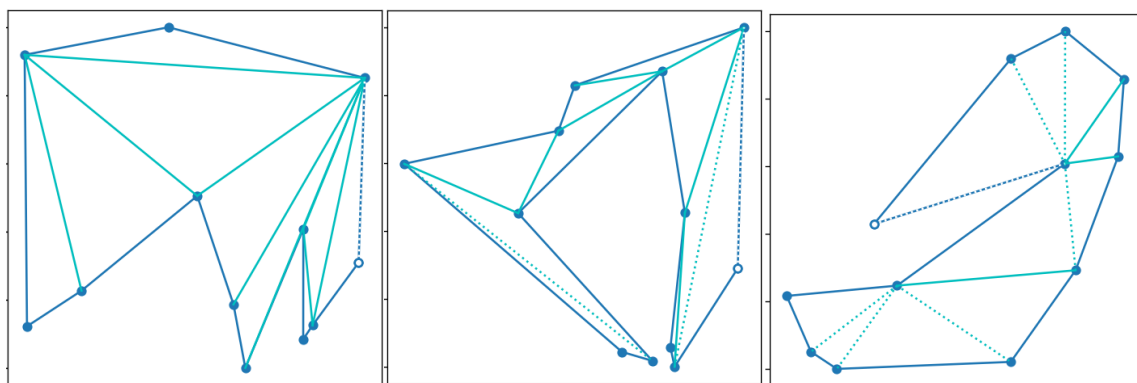


Fig 5 to 7.

The baseline, simple, and flood algorithm's outputs, in that order.
 White dotted vertex and edge indicate first of each. Dotted cyan edges
 indicate removed edges.

Discussion & Conclusion

In the previous results, the average edge count may be higher, as runs with an error are cut short, but still have their edges up to that point recorded. This is also the reason why the baseline edge counts do not line up with the $n-3$ rule. Still, it is a good indicator of how far the algorithm got before encountering an error.

From the results, the two algorithms clearly show that they are able to use less edges, while not increasing the runtime significantly. This is because the simple algorithm runs a short piece of code at the end, while the flood algorithm does both edge addition and reduction at the same time. It is also because of this that the flood algorithm is able to save more edges than the simple algorithm, showing about a 40% improvement over the baseline.

The algorithms are not without its faults, however. As the number of vertices increases, the runtime increases and so does the odds of encountering a fatal error, likely due to floating point errors. In particular, the failure rate for the flood algorithm couldn't be measured, as while in the other algorithms running out of ears is a failure, for it, running out of ears is the signal to stop the current loop.

All in all, the flood algorithm is a useful tool to quickly generate convex subdivisions without too much hassle. Future work may focus on reducing the error rate, or finding even more edges that can be removed.

References

[1] H. Guggenheimer, "The Jordan Curve Theorem and an Unpublished Manuscript by Max Dehn," 1 February 1977. [Online]. Available: <https://webhomes.maths.ed.ac.uk/~v1ranick/jordan/guggenheim.pdf>. [Accessed 05 May 2026].

